

Basics Git/GitHub Workshop: Hands-On Lab

Beginner's Tutorial for Git and GitHub - By Daniel Tyukov

Learn the essentials of Git and GitHub workflows and version control basics for Honors

Workshop Duration: 45 Minutes

Prerequisites: Basic Command Line Familiarity

Tools Required: Git, Python, Text Editor (e.g., VS Code)

*"Just remember: the answer to life, the universe, and everything in your repo is always
'commit #42' — so 'Don't Panic' in Git!"*

Contents

1	Introduction and Installation	3
1.1	Objective	3
1.2	Installing Git	3
1.3	Global Git Configuration	3
2	Configuring SSH with GitHub	4
2.1	Windows	4
2.2	Ubuntu 22.04	4
3	Starting a Project Locally	5
3.1	Objective	5
3.2	Detailed Steps	5
3.3	Result	7
4	Working with GitHub	8
4.1	Objective	8
4.2	Detailed Steps	8
4.3	Result	9
5	Branching and Merging	9
5.1	Result	10
6	Exploring Commits and Recovery (Overview)	11
6.1	Objective	11
6.2	Deatiled Steps	11
7	Advanced Topics (Will Be Demonstrated (If Time Allows))	12
A	Full Dino Game Code	15

1 Introduction and Installation

1.1 Objective

The goal of this section is to ensure everyone have Git installed and configured, and they understand the basic states of Git (Working Directory, Staging Area, Repository). Make sure you have

1.2 Installing Git

Windows: Download Git from <https://git-scm.com/download/win>. Install with default options. Verify installation:

```
1 git --version
```

Ubuntu: Run the following commands in the terminal:

```
1 sudo apt update
2 sudo apt install git -y
3 git --version
```

1.3 Global Git Configuration

Set up your username and email (required for commits):

```
1 git config --global user.name "Your_Name"
2 git config --global user.email "your_email@example.com"
3 git config --global init.defaultBranch main
4 git config --list
```

2 Configuring SSH with GitHub

Why SSH?

Using SSH keys allows you to securely connect to GitHub without entering your username and password every time. Once set up, you can push, pull, and clone from GitHub easily.

2.1 Windows

Prerequisite: You should have Git installed and be using Git Bash (installed with Git for Windows).

1. **Generate an SSH Key:** In Git Bash, run:

```
1 ssh-keygen -t ed25519 -C "your_email@example.com"
```

Press Enter to accept defaults and create the key. If prompted, set a passphrase (optional but recommended).

2. **Find Your Public Key:**

```
1 cat ~/.ssh/id_ed25519.pub
```

Copy the entire output (this is your public key).

3. **Add the Key to GitHub:**

- (a) Go to <https://github.com> and sign in.
- (b) Click on your profile picture > Settings > SSH and GPG keys > New SSH key.
- (c) Give it a title (e.g. "My Laptop") and paste the public key into the "Key" field.
- (d) Click "Add SSH key".

4. **Test the Connection:**

```
1 ssh -T git@github.com
```

You may be asked to confirm the host. Type **yes**. If successful, you should see a message like: *"Hi username! You've successfully authenticated, but GitHub does not provide shell access."*

2.2 Ubuntu 22.04

Prerequisite: Ensure Git is installed:

```
1 sudo apt update
2 sudo apt install git -y
```

1. **Generate an SSH Key:**

```
1 ssh-keygen -t ed25519 -C "your_email@example.com"
```

Press **Enter** at the prompts to accept defaults. Set a passphrase if desired.

2. Retrieve Your Public Key:

```
1 cat ~/.ssh/id_ed25519.pub
```

Copy the output.

3. Add the Key to GitHub:

- (a) Log into [GitHub](#).
- (b) Click on your profile picture > **Settings** > **SSH and GPG keys** > **New SSH key**.
- (c) Enter a title, paste the public key, and click "Add SSH key".

4. Test the Connection:

```
1 ssh -T git@github.com
```

Type **yes** if prompted. A successful test shows a message: *"Hi username! You've successfully authenticated, but GitHub does not provide shell access."*

That's it!

You've now configured SSH keys on both Windows and Ubuntu 22.04. From this point on, you can use Git commands like **git clone**, **git push**, and **git pull** using the SSH URLs from GitHub without entering your password each time.

3 Starting a Project Locally

3.1 Objective

Setup your local environment, create the repository locally, and start coding.

3.2 Detailed Steps

1. Create a Directory and Open in VS Code:

- In your terminal:

```
1 mkdir dino_game_workshop
2 cd dino_game_workshop
```

- Open this folder in VS Code: In VS Code, go to **File** > **Open Folder** (Windows/Linux) then select the **dino_game_workshop** folder.

Alternatively, if you have the VS Code command-line tool set up, you can run:

```
1 code .
```

2. Initialize a Git Repository:

- In the VS Code integrated terminal (**View > Terminal**), run:

```
1 git init
```

This creates a new `.git` folder that tracks your repository's history.

3. **Create an Incomplete Game File:** We will create a partial, incorrect version of the `dino_game.py` file. Later, we'll add the correct code in a new branch, causing a merge conflict. For now, just add a small portion of the code or alter it so it won't run correctly. For example, remove the obstacle logic and leave a placeholder for it:

In VS Code, create a new file `dino_game.py` and add:

```
1 import curses
2 import time
3
4 def main(stdscr):
5     curses.curs_set(0)
6     stdscr.nodelay(1)
7     stdscr.timeout(100)
8
9     # Initialize variables (intentionally leave out 'random'
10    ↪ and 'obstacles' for now)
11    sh, sw = stdscr.getmaxyx()
12    w = sw // 2
13    dino = [sh - 2, w]
14    score = 0
15    jump = False
16    jump_height = 5
17    jump_count = 0
18
19    # NOTE: Obstacle logic is missing here intentionally
20    # We'll add it later on another branch, causing a merge
21    ↪ conflict.
22
23    while True:
24        stdscr.clear()
25
26        # Display Dino (using "D", but in the future we might
27        ↪ change this character)
28        stdscr.addstr(dino[0], dino[1], "D")
29
30        # For now, no obstacle logic. Just increment score.
31        score += 1
32        stdscr.addstr(0, 0, f"Score: {score}")
33
34        # Handle jump logic (no changes needed here)
35        if jump:
36            if jump_count < jump_height:
37                dino[0] -= 1
38                jump_count += 1
39            else:
40                jump = False
```

```
38         else:
39             if dino[0] < sh - 2:
40                 dino[0] += 1
41             else:
42                 jump_count = 0
43
44         # Check for user input (no obstacle checks here)
45         key = stdscr.getch()
46         if key == ord('q'):
47             break
48         if key == ord(' ') and dino[0] == sh - 2:
49             jump = True
50
51         stdscr.refresh()
52
53 if __name__ == "__main__":
54     curses.wrapper(main)
```

This code omits the `random` import and any references to obstacles. Later, we will introduce obstacles in a new branch and attempt to merge them in, causing a merge conflict with lines that are positioned differently.

4. Check Git Status:

```
1 git status
```

You should see `dino_game.py` as an untracked file.

5. Stage and Commit the File:

```
1 git add dino_game.py
2 git commit -m "Initial commit with incomplete dino game code"
```

This creates your first commit with an intentionally incomplete version of the code.

6. Create a .gitignore File: We'll ignore some files to keep the repository clean.

```
1 echo "*.log" > .gitignore
2 echo "*.txt" >> .gitignore
3 git add .gitignore
4 git commit -m "Add .gitignore"
```

3.3 Result

You now have a Git repository with an initial, incomplete game file committed. By starting with a partial and incorrect version, you're setting the stage for practicing branching and merging. In the next sections, you will create new branches, add the correct code, and handle the merge conflicts that arise.

4 Working with GitHub

4.1 Objective

Connect the local repository to a new GitHub repository and push local changes online, ensuring the local code is fully synced with the remote.

4.2 Detailed Steps

1. Create a New GitHub Repository:

- Go to <https://github.com>, and sign in.
- Click the + icon (top-right), then select **New repository**.
- Name it `dino_game_workshop` (exactly the same as your local folder for consistency).
- Choose **public** or **private** as needed, then click **Create repository**.

You'll see instructions for connecting this new repository to an existing local repository.

2. Add the GitHub Remote to Your Local Repo:

- In VS Code's terminal (ensure you're inside the `dino_game_workshop` directory), run:

```
1 git remote add origin git@github.com:YourUsername/  
  ↪ dino_game_workshop.git
```

Replace `YourUsername` with your GitHub username.

3. Push Your Local Commits to GitHub:

- Push your local changes to the remote repository with:

```
1 git push -u origin main
```

This uploads your local commits to the `main` branch of the GitHub repository.

- After this command, you can go back to the GitHub page and refresh to see your files online.

4. Verify Your Remote Setup:

- On GitHub, confirm that `dino_game.py` and `.gitignore` are listed.
- This ensures your local repository is now successfully synced with GitHub.

5. Clone the Repository (For Verification):

- If you want to verify that anyone with access can clone this repository:

```
1 cd ..  
2 git clone git@github.com:YourUsername/dino_game_workshop.  
  ↪ git dino_game_clone
```

- Now you have a second copy of the repository. Open it to confirm it has the same files and history.

4.3 Result

Your local work is now backed up on GitHub. You can continue to develop locally, commit changes, and `git push` them to keep GitHub updated. In the next sections, you will work with branches, merges, and pull operations.

5 Branching and Merging

Objective

Create a new branch on GitHub with the correct game code, merge it into `main` (which currently has incorrect code), and resolve the resulting merge conflict in VS Code. We will cause a conflict by making a change in `main` before merging.

Detailed Steps

1. Create and Switch to a New Feature Branch:

```
1 git checkout -b feature-obstacles
```

You are now on the `feature-obstacles` branch.

2. Replace `dino_game.py` with the Correct Code: In VS Code, open `dino_game.py`. Remove all previous code and paste the full correct code from the **APPENDIX**. Save the file.

3. Stage and Commit the Correct Code:

```
1 git add dino_game.py
2 git commit -m "Add full Dino game functionality with
  ↳ obstacles"
```

Push this branch to GitHub:

```
1 git push -u origin feature-obstacles
```

Now you have `feature-obstacles` on GitHub with the correct code, and `main` on GitHub with the incorrect.

4. Ensure `main` is Up to Date: From your `main` branch (already on GitHub), modify `dino_game.py` to cause a conflict. For example, change the character displayed for the dino from "D" to "@" in `main`:

```
1 # In main branch dino_game.py (incomplete code):
2 stdscr.addstr(dino[0], dino[1], "@") # Changed from "D" to "
  ↳ @"
```

Then:

```
1 git add dino_game.py
2 git commit -m "Change Dino character to @ in main"
3 git push origin main
```

5. Merge the Feature Branch into main: Switch back to main:

```
1 git checkout main
2 git pull origin main
3 git merge feature-obstacles
```

A merge conflict occurs because `main` and `feature-obstacles` modify the same line differently ("`@`" in `main` vs. "`D`" in `feature-obstacles`).

6. Resolve the Merge Conflict in VS Code:

- Open `dino_game.py` in VS Code.
- Look for conflict markers `<<<`, `===`, `>>>`.
- Choose the correct code from `feature-obstacles` (the full code with "`D`").
- Remove the conflict markers.
- Save the file.

7. Finalize the Merge:

```
1 git add dino_game.py
2 git commit
3 git push origin main
```

Now the `main` branch has the correct, fully functional code merged in.

8. Optional: Abort the Merge if Needed:

```
1 git merge --abort
```

Use only if you want to cancel the merge attempt and start over.

5.1 Result

You have successfully created a branch with the correct code, caused a deliberate merge conflict by changing the dino character in `main`, and resolved the conflict in VS Code. This simulates a common real-world scenario, giving you practice in branch management and conflict resolution.

6 Exploring Commits and Recovery (Overview)

6.1 Objective

Learn how to view commit history, revert unwanted changes, reset to previous states, and understand basic rebasing.

6.2 Detailed Steps

1. View History:

```
1 git log --oneline
```

Shows abbreviated commit history. Identify a commit by its hash.

2. Check Out a Previous Commit (Read-Only):

```
1 git checkout <commit-hash>
```

Explore the project at that state. **Note:** You are now in a "detached HEAD" state. Return to main:

```
1 git checkout main
```

3. Revert a Commit:

```
1 git revert <commit-hash>
```

Creates a new commit that undoes the changes from the specified commit. Use this when you want a clean history without removing existing commits.

4. Reset to a Previous Commit:

```
1 git reset --hard <commit-hash>
```

Moves the current branch pointer back to <commit-hash> and discards all changes after it. **Warning:** This is destructive, lost commits are hard to recover without backups.

5. Rebase for a Cleaner History:

```
1 git rebase -i <commit-hash>
```

Allows you to reorder, squash, or edit commits before <commit-hash>. **Example:** Squashing multiple small commits into one for a cleaner history.

7 Advanced Topics (Will Be Demonstrated (If Time Allows))

- Creating pull requests on GitHub.
- Tagging and creating releases:

```
1 git tag v1.0.0  
2 git push origin v1.0.0
```

Git Cheatsheet

A quick reference guide for common Git commands.

Git Basics

- `git init` - Initialize a Git repository.
- `git clone <url>` - Clone a repository.
- `git status` - Check the status of your repository.
- `git help <command>` - Get help for a command.

Configuration

- `git config --global user.name "Your Name"` - Set global username.
- `git config --global user.email "you@example.com"` - Set global email.
- `git config --list` - View all configuration settings.

Working with Changes

- `git add <file>` - Stage a file.
- `git add .` - Stage all files.
- `git commit -m "Message"` - Commit changes with a message.
- `git commit --amend` - Amend the last commit.

- `git diff` - Show unstaged changes.
- `git diff --staged` - Show staged changes.

Branching

- `git branch` - List branches.
- `git branch <name>` - Create a new branch.
- `git checkout <branch>` - Switch to a branch.
- `git checkout -b <name>` - Create and switch to a branch.
- `git branch -d <name>` - Delete a branch.
- `git merge <branch>` - Merge a branch into the current branch.

Remote Repositories

- `git remote add origin <url>` - Add a remote repository.
- `git remote -v` - List remote repositories.
- `git fetch` - Fetch updates from a remote.

- `git pull` - Fetch and merge updates.
- `git push` - Push changes to a remote.
- `git push -u origin <branch>` - Push and set upstream.

Undo Changes

- `git checkout <file>` - Discard changes in a file.
- `git reset HEAD <file>` - Unstage a file.
- `git revert <commit>` - Revert a commit.
- `git reset --soft <commit>` - Reset to a commit, keep changes staged.
- `git reset --hard <commit>` - Reset to a commit, discard changes.

Stashing

- `git stash` - Stash changes.
- `git stash list` - List stashes.
- `git stash apply` - Apply the latest stash.
- `git stash drop` - Remove a stash.

Logs and History

- `git log` - View commit history.
- `git log --oneline` - View history in one line per commit.
- `git log --graph` - View history as a graph.
- `git blame <file>` - Show who changed each line.
- `git config --global alias.co checkout` - Alias for checkout.
- `git config --global alias.br branch` - Alias for branch.
- `git config --global alias.st status` - Alias for status.

Tags

- `git tag` - List tags.
- `git tag <name>` - Create a tag.
- `git show <tag>` - Show tag details.
- `git push origin <tag>` - Push a tag to a remote.
- Use `.gitignore` to exclude files.
- Use `git reflog` to recover lost commits.
- Add `-n` or `--dry-run` to preview changes before executing.

Tips

Collaboration

- `git pull origin main` - Pull changes from main.
- `git push origin main` - Push changes to main.
- `git fetch --all` - Fetch updates from all remotes.
- `git cherry-pick <commit>` - Apply a specific commit.

Aliases

For more information, visit the [official Git documentation](#).

A Full Dino Game Code

```
1 import curses
2 import random
3 import time
4
5 def main(stdscr):
6     curses.curs_set(0)
7     stdscr.nodelay(1)
8     stdscr.timeout(100)
9
10    # Initialize variables
11    sh, sw = stdscr.getmaxyx()
12    w = sw // 2
13    dino = [sh - 2, w]
14    obstacles = []
15    score = 0
16    jump = False
17    jump_height = 5
18    jump_count = 0
19
20    while True:
21        stdscr.clear()
22
23        # Display Dino
24        stdscr.addstr(dino[0], dino[1], "D")
25
26        # Add obstacles
27        if random.randint(0, 10) == 0:
28            obstacles.append([sh - 2, sw - 1])
29
30        # Move obstacles and detect collision
31        for obstacle in obstacles[:]:
32            obstacle[1] -= 1
33            if obstacle[1] == dino[1] and obstacle[0] == dino[0]:
34                stdscr.addstr(sh // 2, sw // 2 - 5, "GAME OVER!")
35                stdscr.refresh()
36                time.sleep(2)
37                return
38            if obstacle[1] < 0:
39                obstacles.remove(obstacle)
40
41        # Display obstacles
42        for obstacle in obstacles:
43            stdscr.addstr(obstacle[0], obstacle[1], "X")
44
45        # Handle jump
46        if jump:
```

```
47         if jump_count < jump_height:
48             dino[0] -= 1
49             jump_count += 1
50         else:
51             jump = False
52     else:
53         if dino[0] < sh - 2:
54             dino[0] += 1
55         else:
56             jump_count = 0
57
58     # Display score
59     score += 1
60     stdscr.addstr(0, 0, f"Score:_{score}")
61
62     # Check for user input
63     key = stdscr.getch()
64     if key == ord('q'):
65         break
66     if key == ord('_') and dino[0] == sh - 2: # Jump if on
67         ↪ the ground
68         jump = True
69
70     stdscr.refresh()
71
72 if __name__ == "__main__":
73     curses.wrapper(main)
```